FULL STACK DEVELOPER CASE STUDY

OpsPortal

Mini ERP + CRM Operations Portal
Technical Documentation

Project	OpsPortal — wholesale & distribution ERP/CRM
Backend	Django 5.2.7 · Django REST Framework 3.16 · PostgreSQL 16
Frontend	React 19 · TypeScript 5.9 · Vite 7
Infrastructure	Docker Compose · nginx · gunicorn · GitHub Actions
Test suite	168 automated tests · 79.6% line coverage
API surface	42 documented paths · 67 operations · OpenAPI 3
Author	Kashish Shrivastav
Repository	github.com/kashish2210/OpsPortal
Date	12 August 2026

Contents

1 **Overview and scope**	What the system does, and what was deliberately left out
2 **Technology stack**	Every dependency and why it is there
3 **Architecture**	Containers, request routing, and the single-origin decision
4 **Data model**	Five apps, eight tables, and the snapshot columns
5 **Core modules**	Auth & roles, CRM, inventory, sales challans
6 **Business rules**	The stock engine, concurrency safety, and immutability
7 **Authentication and authorisation**	Two login surfaces, one session; the role matrix
8 **API reference**	Conventions, the error envelope, and every endpoint
9 **Bonus features**	PDF export, Cloudinary media, SMTP, CI/CD
10 **Testing**	What is covered, and how to run it
11 **CI/CD pipeline**	Five jobs, PR reporting, and local runs with act
12 **Deployment**	Server setup, environments, and hosting options
13 **Configuration reference**	Every environment variable
14 **Assumptions**	Decisions taken where the brief was silent
15 **Known limitations**	What is not built, and what would come next
A **Command reference**	Everything you need to run and operate the project

How to read this document

Sections 1–5 describe what was built. Section 6 is the part worth reading closely — it covers the stock and challan rules the brief calls out, and the concurrency guarantees behind them. Sections 10–13 are operational.

1 Overview and scope

OpsPortal is a small ERP/CRM system for a wholesale and distribution company. It covers the day-to-day operations of three internal teams — sales, warehouse and accounts — across customers, products, stock and outbound deliveries.

The brief asked for four core modules. All four are complete, and the emphasis throughout has been on the business rules rather than on breadth: stock that cannot go negative, documents that stay immutable once issued, and an audit trail that explains every change.

Delivered

Module	Scope	Status
Authentication & roles	Four roles, JWT for the API, Django sessions for the server-rendered pages, 16 auth screens	Complete
Customer CRM	Add / edit / search / detail, dated follow-up notes, lead pipeline	Complete
Products & inventory	Catalogue plus an append-only stock ledger with balances	Complete
Sales challans	Multi-line documents, auto numbering, draft / confirmed / cancelled with stock deduction	Complete
REST API	Validation, status codes, uniform errors, pagination, search and filtering	Complete
Frontend	React SPA, responsive, light and dark themes	Complete
Bonus	Challan PDF export, Cloudinary media, SMTP, Docker, CI/CD	Complete

Out of scope

Purchase orders and invoices appear in the brief's scene-setting paragraph but not in its list of required modules, so they are not built. That was a deliberate reading: the four named modules were taken to their proper depth instead of adding two more shallow ones.

Scale of the implementation

Area	Files	Lines
Backend application code	58	6,682
Backend tests	7	1,875
Frontend TypeScript / React	32	6,119
Django templates (auth screens)	25	776
Shared stylesheet	2	1,804

> **A note on the technology choice**
> The brief specifies Node.js with TypeScript and Express or NestJS. This implementation uses **Django and Django REST Framework** instead, at the reviewer's explicit request, together with Django's built-in auth for the server-rendered login pages. Every other requirement is met as written, and the frontend is React with TypeScript, so the TypeScript requirement holds there.

2 Technology stack

Every dependency is pinned. Nothing is included that is not actually used.

Backend

Package	Version	Why
Django	5.2.7 LTS	ORM, migrations, auth, admin, template engine
djangorestframework	3.16.1	REST API layer, serializers, viewsets
djangorestframework-simplejwt	5.5.1	JWT issue / refresh / blacklist
psycopg[binary]	3.2.10	PostgreSQL driver
django-filter	25.2	Declarative query-string filtering
django-cors-headers	4.9.0	CORS for split-origin development
django-environ	0.12.0	Typed environment configuration
drf-spectacular	0.28.0	OpenAPI 3 schema, Swagger UI and ReDoc
reportlab	4.4.4	Challan PDF generation, no system libraries needed
cloudinary + storage	1.44.1 / 0.3.0	Product image CDN
django-storages + boto3	1.14.6 / 1.40.29	Alternative S3 / R2 / MinIO backend
whitenoise	6.11.0	Hashed, compressed static files
gunicorn	23.0.0	WSGI server
pillow	11.3.0	Image validation and processing

Frontend

Package	Version	Why
react / react-dom	19.1.1	UI runtime
react-router-dom	7.9.1	Client-side routing and route guards
typescript	5.9.2	Strict mode, no implicit any
vite	7.1.5	Dev server with API proxy, production bundler

No component library, state manager or data-fetching library is used. The UI is built on a single hand-written stylesheet and a small set of shared components; server state is handled by a purpose-built hook with request cancellation. This keeps the bundle at roughly 99 KB gzipped and the dependency surface small.

Development and CI

Tool	Purpose
Docker Compose	The whole stack in one command
coverage.py	Line and branch coverage, reported on every pull request
unittest-xml-reporting	JUnit XML so CI can annotate failing tests on the diff

Tool	Purpose
ruff	Linting and formatting, enforced in CI
Mailpit	Local SMTP server with a web inbox
act	Runs the GitHub Actions workflows locally

3 Architecture

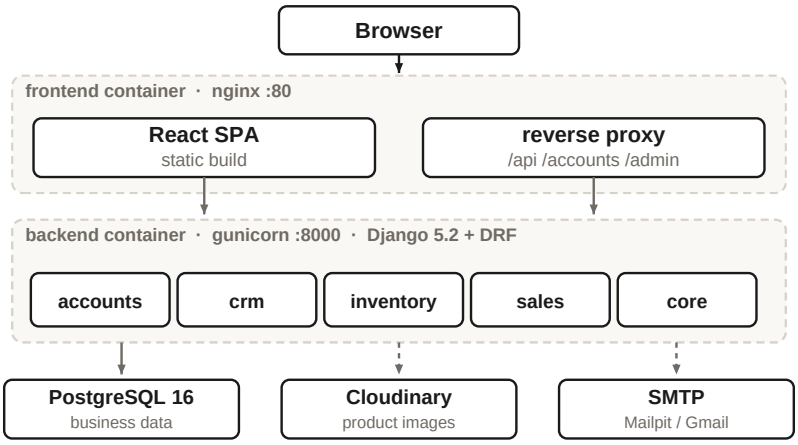


Figure 1 — container topology. Solid arrows are always-on; dashed arrows are optional integrations.

The single-origin decision

nginx fronts both the React bundle and Django. Every request arrives on one origin and is routed by path, which has three consequences worth stating explicitly.

- The session cookie Django sets on its login page is also sent to `/api/auth/session-token/`. That is what makes the session-to-JWT handoff possible at all.
- CORS and CSRF friction disappears — there is no cross-origin request to configure.
- The Vite dev server proxies the same five path prefixes, so development and production behave identically rather than diverging in ways that only surface after deployment.

Request routing

Path	Handled by	Notes
/	React SPA	History fallback to index.html
/assets/*	nginx, from disk	Content-hashed, cached for a year
/api/*	Django REST Framework	JWT bearer authentication
/accounts/*	Django auth views	Session cookie, server-rendered
/admin/*	Django admin	Staff only
/static/*	WhiteNoise, via Django	Hashed filenames
/media/*	nginx, from a shared volume	Only when media is stored locally

> **Why /media/ is served by nginx rather than proxied**
> Django only serves `MEDIA_URL` when `DEBUG=True`. Proxying it would therefore return 404 in the production image. The media volume is mounted read-only into the nginx container and served straight from disk. With Cloudinary enabled the URLs point at the CDN and never reach nginx at all.

Backend layering

Views are deliberately thin. The rules that must never break live in service modules that are called from the API, the seed command and the tests alike, so there is exactly one implementation of each rule.

App	Responsibility
core	Pagination, role permissions, the error envelope, audit base models, gap-free document numbering, dashboard aggregation, demo seeding
accounts	Custom user with a role, JWT endpoints, and the complete server-rendered auth screen set
crm	Customer and FollowUp
inventory	Product, StockMovement, and the stock engine
sales	SalesChallan, ChallanItem, the state machine and PDF rendering

Frontend structure

```
src/
  api/ fetch client with silent JWT refresh, typed endpoints, API types
  auth/ AuthContext, route guards, client-side role matrix
  components/ layout, shared UI kit, toasts, theme toggle
  hooks/ data fetching with cancellation, debounce
  pages/ one folder per module
  styles/ sketch.css — the shared design system
```

4 Data model

Eight tables across five apps. Relationships are shown with the deletion behaviour, because that is where the audit guarantees actually live.

Table	Key fields	Notes
accounts_user	username, email (unique), role, is_active	One role per user: ADMIN, SALES, WAREHOUSE or ACCOUNTS
crm_customer	code (CUST-00001), name, mobile (unique), business_name, gst_number, customer_type, status, follow_up_date	Never deleted — archived by setting status to INACTIVE
crm_followup	customer, note, channel, next_follow_up_date, created_by	Writing one rolls the customer's next-contact date forward
inventory_product	sku (unique), name, category, unit_price, current_stock, min_stock_alert, location, image	current_stock is a running balance maintained only by the ledger
inventory_stockmovement	product, movement_type (IN/OUT), quantity, reason, reference, balance_after, created_by	Append-only. Not editable or deletable, including in Django admin
sales_saleschallan	challan_number (CH-YYYYMM-00001), customer, status, customer_* snapshot columns, totals, audit columns	PROTECT on customer, so a customer with challans cannot be removed
sales_challanitem	challan, product, product_* snapshot columns, quantity, unit_price, line_total	PROTECT on product
core_sequence	key, last_value	Locked counter behind document numbering

Why the snapshot columns exist

A challan line copies the product's name, SKU, category, unit and price at the moment it is saved, and the challan copies the customer's billing details. Renaming or repricing a product later must never rewrite a document that has already been issued to a customer. The brief asks for this explicitly, and there is a test that renames and reprices a product then asserts the challan totals are unchanged.

5 Core modules

5.1 Authentication and roles

Four roles matching the teams named in the brief. A single role per user keeps permission checks cheap and matches how a company of this size actually works.

Sixteen server-rendered screens cover the whole account lifecycle: login, signup, signup-closed, logout confirmation, signed-out, profile, password change and done, password set, the four-step password reset, e-mail management, e-mail confirmation, verification sent, verification required, and account inactive.

5.2 Customer CRM

Field	Rule
Customer name	Required, at least two characters
Mobile number	Required and unique — it is how field staff look customers up
E-mail	Optional, normalised to lower case
Business name	Optional
GST number	Required for wholesale and distributor; optional for retail. Validated against the 15-character GSTIN format
Customer type	Retail, Wholesale or Distributor
Address	Street, city, state and pincode
Status	Lead, Active or Inactive
Follow-up date	Required when creating a lead, so nothing is forgotten
Notes	Free text, plus a separate dated follow-up log

Search covers name, business name, mobile, e-mail, customer code, GSTIN and city in a single query parameter. Filters exist for status, type, city, GST presence and follow-up state (due, upcoming or unscheduled).

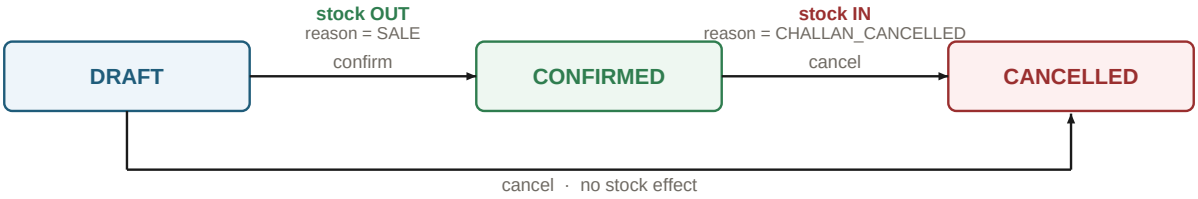
5.3 Products and inventory

Products carry the fields the brief lists — name, SKU, category, unit price, current stock, minimum stock alert and location — plus unit of measure, HSN code and GST rate, which a distribution business needs in practice.

The stock movement log records exactly what the brief asks for: the product, the quantity changed, whether it was IN or OUT, the reason, who did it and when. It also stores the resulting balance, so the ledger and the running total can be reconciled at any point.

5.4 Sales challans

A sales user selects a customer, adds multiple products with quantities, and saves the challan as a draft or confirms it immediately. Challan numbers are generated automatically in the form CH-YYYYMM-00001, restarting each month.



A draft reserves nothing. Confirmed documents are immutable — edit and delete both return HTTP 400.

Figure 2 — challan state machine and the stock movements each transition produces.

6 Business rules

This section covers the parts of the brief that are easy to implement naively and hard to implement correctly.

6.1 Stock can never go negative

Every change to a product's stock goes through one module. It locks the product rows before reading them, so two sales users confirming challans at the same instant serialise on the lock instead of both passing the availability check.

```
# inventory/services.py
products = Product.objects.select_for_update()
                  .filter(pk__in=sorted(product_ids)) # sorted → no deadlock
                  .order_by('pk')

# every short line is collected before raising, not just the first
if shortages:
    raise InsufficientStockError(message, details={'shortages': shortages})
```

Products are locked in a deterministic order, which removes the classic deadlock where two transactions grab the same two rows in opposite orders.

6.2 All-or-nothing, with every shortage reported

A challan with three lines where two are short does not partially ship. The engine collects every shortage before raising, so the interface can highlight all the bad lines at once rather than making the user fix them one at a time.

```
HTTP 400
{
  "success": false,
  "error": {
    "code": "insufficient_stock",
    "message": "Insufficient stock for: SF-GL-CR (need 10, have 0).",
    "details": {
      "shortages": [{
        "product_id": 17, "sku": "SF-GL-CR",
        "product_name": "Cut-resistant Gloves (Pair)",
        "requested": 10, "available": 0, "short_by": 10
      }]
    }
  }
}
```

6.3 Drafts reserve nothing

Stock moves only on confirmation. Creating a challan with `confirm: true` is all-or-nothing as well — a stock failure rolls the challan back too, so a rejected request never leaves an orphaned draft behind. There is a test asserting the challan count is unchanged after a failed save-and-confirm.

6.4 Cancelling is compensating, not destructive

Cancelling a confirmed challan writes IN movements with the reason CHALLAN_CANCELLED and returns the stock. Cancelling a draft touches nothing. The ledger stays a faithful record of what actually happened rather than being edited after the fact — which is the whole point of an append-only log.

6.5 Confirmed documents are immutable

Only drafts can be edited or deleted. A confirmed challan must be cancelled and re-raised. This is enforced in three places — the serializer, the service layer and Django admin — because a rule enforced in only one of them is a rule that will eventually be bypassed.

6.6 Document numbers do not collide

`MAX(id) + 1` breaks the moment two users save at the same instant. Numbering uses a counter table locked with SELECT … FOR UPDATE, so concurrent requests serialise on the counter instead of colliding on a unique index.

6.7 Stock is a ledger, not a field

The product's stock figure is a running balance; the movement log explains every change to it and stores the balance after each one. A reconciliation helper can rebuild the balance from the ledger to prove the two agree.

7 Authentication and authorisation

There are two ways in. They share one user table, one password and one session.

Route A — signing in inside the application

The React login posts to `/api/auth/login/` and stores a JWT pair. Access tokens last 60 minutes; the client refreshes them silently on a 401 and replays the original request. Refresh tokens rotate, and the previous one is blacklisted on use.

Route B — signing in on the Django page

The server-rendered pages use Django sessions. After signing in, the profile page offers an **Open OpsPortal** button that lands on a handoff route, which calls `/api/auth/session-token/`. That endpoint authenticates with the session cookie and returns a JWT pair, so the SPA picks the session up without a second login.

Both routes accept a username or an e-mail address.

Role permission matrix

Module	Admin	Sales	Warehouse	Accounts
Customers and follow-ups	read + write	read + write	read	read
Products	read + write	read	read + write	read
Stock movements	read + write	read	read + write	read
Sales challans	read + write	read + write	read	read
Challan PDF download	yes	yes	yes	yes
Users and roles	full	—	—	—

The matrix is declared once per viewset and mirrored in the frontend purely to decide what to *offer*. The server is always the enforcer: a crafted request still returns 403, and there are tests for each denial.

> **Accounts can download PDFs but cannot edit anything**
> The accounts team needs the paperwork without the ability to change documents. That is why the PDF endpoint is readable by every role while every write endpoint is not — a small detail, but it is the difference between a role model that fits the business and one that merely looks tidy.

Security measures

- Passwords hashed with Django's default PBKDF2, and validated against length, similarity, common-password and numeric-only rules.
- Refresh-token rotation with blacklisting, so a stolen refresh token is single-use.
- Login throttled to 20 requests a minute; password reset to 10 an hour.
- Emailed reset links are built from configuration, not the request's Host header, so a spoofed or misconfigured proxy header cannot redirect them. Tested explicitly.
- ALLOWED_HOSTS rejects unknown hosts before any view runs.
- Uploads are validated by Pillow for real image content and restricted by size and format.
- Accounts are deactivated rather than deleted, preserving the audit trail.
- HSTS, secure cookies and SSL redirect switch on automatically when DEBUG is off.

8 API reference

42 documented paths, 67 operations. Interactive documentation is generated from the code and served at `/api/docs/`; a Postman collection with 67 requests ships in the repository.

Conventions

Every list endpoint is paginated with the same envelope:

```
{ "count": 137, "page": 2, "page_size": 20, "total_pages": 7,
  "next": "…", "previous": "…", "results": [ … ] }
```

Every error uses one envelope, so the client parses a single shape:

```
{ "success": false,
  "error": { "code": "validation_error",
             "message": "Human readable summary",
             "details": { "field": ["reason"] } } }
```

Status	Meaning
200 / 201 / 204	Success, created, deleted
205	Logged out — refresh token blacklisted
400	Validation failure or business-rule violation
401	Missing or expired token
403	Authenticated, but the role does not permit it
404	Not found
429	Throttled

Authentication

Method	Path	Purpose
POST	/api/auth/login/	Username or e-mail and password → JWT pair
POST	/api/auth/refresh/	Rotate the access token
POST	/api/auth/logout/	Blacklist the refresh token
GET / PATCH	/api/auth/me/	Current user; PATCH edits own details, not the role
POST	/api/auth/change-password/	Change own password
POST	/api/auth/session-token/	Django session → JWT pair

Customers

Method	Path	Purpose
GET / POST	/api/customers/	List with search and filters; create
GET / PATCH / DELETE	/api/customers/{id}/	Detail; edit; archive (sets INACTIVE)
GET / POST	/api/customers/{id}/follow-ups/	Follow-up history; record one

Method	Path	Purpose
GET	/api/customers/options/	Slim picker list for the challan builder
GET	/api/customers/stats/	CRM counters
GET	/api/follow-ups/	Follow-ups across all customers

Filters: `q, status, customer_type, city, has_gst, follow_up=due|upcoming|none, created_after, created_before`.

Products and stock

Method	Path	Purpose
GET / POST	/api/products/	List; create (opening_stock logs an OPENING movement)
GET / PATCH / DELETE	/api/products/{id}/	Detail; edit; archive
POST / DELETE	/api/products/{id}/image/	Upload or remove the product photo
GET	/api/products/{id}/movements/	Ledger for one product
POST	/api/products/{id}/adjust-stock/	Stock-take correction to an absolute figure
GET	/api/products/low-stock/	At or below the alert level
GET	/api/products/categories/ · /locations/	Distinct values for dropdowns
GET / POST	/api/stock-movements/	The full ledger; record a movement

Filters: `q, category, location, stock=low|out|in, is_active, movement_type, reason, reference, created_after, created_before`.

Sales challans

Method	Path	Purpose
GET / POST	/api/challans/	List; create (confirm: true saves and confirms)
GET / PATCH / DELETE	/api/challans/{id}/	Detail; edit or delete drafts only
POST	/api/challans/{id}/confirm/	Deduct stock and confirm
POST	/api/challans/{id}/cancel/	Cancel; restores stock if it was confirmed
GET	/api/challans/{id}/pdf/	Download the challan as a PDF
GET	/api/challans/{id}/stock-movements/	Movements this challan produced

Administration and utility

Method	Path	Purpose
GET / POST / PATCH	/api/users/	Employee administration (admin only)
POST	/api/users/{id}/activate/ · /deactivate/	Toggle account access
GET	/api/dashboard/	All KPI counters and recent activity in one call
GET	/api/health/	Liveness probe, public, checks the database

Method	Path	Purpose
GET	/api/schema/ · /docs/ · /redoc/	OpenAPI 3 schema and interactive docs

9 Bonus features

All four bonus items in the brief are implemented, plus a real SMTP server for local development.

9.1 Challan PDF export

A proper A4 delivery-challan document rendered with ReportLab — chosen over an HTML-to-PDF converter so the production image needs no cairo or pango system libraries and stays slim.

- Renders from the stored snapshot columns, so a reprint months later reproduces the document exactly as it was issued.
- Indian digit grouping (12,34,567.50) and rupees in words, both of which Indian delivery documents require.
- Drafts and cancelled challans are watermarked diagonally so a draft cannot be mistaken for a live document.
- Company details, GSTIN and address come from configuration.
- Available to every role, including read-only Accounts.

9.2 Product images on a CDN

Media storage is selected by a single environment variable with three options, and switching between them requires no code change because everything is written through Django's storage API.

MEDIA_STORAGE	Backend	Notes
local	Filesystem volume	Default. Zero setup, works offline
cloudinary	Cloudinary CDN	25 GB free, image transformations, currently active
s3	S3-compatible	AWS S3, Cloudflare R2, Backblaze B2 or MinIO

Uploads are validated for real image content, restricted to JPEG, PNG and WebP under 5 MB, namespaced by SKU, and given a unique key so replacing an image never collides with a cached copy. Replacing or deleting an image removes the previous object rather than orphaning it.

9.3 SMTP

Docker Compose runs Mailpit, a real SMTP server with a searchable web inbox on port 8025. Password-reset and verification e-mails can be walked through end to end without a mail provider and without anything leaving the machine. Switching to Brevo, SendGrid, SES or Gmail is an environment change; ready-made blocks for each are in the example file.

9.4 Docker

One command builds both images, starts PostgreSQL, waits for it to become healthy, runs migrations, collects static files, seeds demo data and starts gunicorn. The backend image runs as a non-root user; the frontend is a two-stage build whose runtime layer contains no Node and no source.

9.5 GitHub Actions

Covered in section 11.

10 Testing

168 automated tests at 79.6% line coverage. The suite is weighted towards the rules that would cause real damage if they broke, rather than spread evenly for the sake of a coverage number.

Area	Tests	What is covered
Stock engine	13	IN/OUT, never-negative, bulk all-or-nothing, duplicate-line merging, archived products, stock-take, ledger rebuild
Challan workflow	40	State machine, stock deduction and restoration, immutability, snapshots, numbering, role enforcement
PDF export	16	Valid output in all three states, snapshot fidelity, money formatting, amount in words, endpoint behaviour
Product images	18	Upload, replace, delete, validation, role enforcement, storage-backend selection
CRM	26	Validation rules, GST requirement, search, filters, follow-ups
Accounts and auth	45	JWT lifecycle, session handoff, role matrix, every rendered auth page
Emailed links	10	Correct origin, port preservation, host-header spoofing

Running the tests

```
# in Docker
docker compose exec backend python manage.py test

# locally
cd backend && python manage.py test

# one module, or one test
python manage.py test sales
python manage.py test sales.tests.ChallanServiceTests.test_confirm_reduces_stock

# with coverage
coverage run manage.py test && coverage report && coverage html

# frontend type safety
cd frontend && npm run typecheck
```

Tests do not touch live services
The image-upload tests force local filesystem storage even when the developer's environment is configured for Cloudinary, so running the suite never uploads to a real account. E-mail uses Django's in-memory backend.

11 CI/CD pipeline

Continuous integration runs on every push and pull request. Five jobs, four of them in parallel.

Job	What it does	Blocking
lint	ruff check and ruff format --check across the backend	Yes
backend-tests	168 tests against a real PostgreSQL 16 service, with coverage and JUnit XML; also asserts there are no missing migrations and that the demo seed still works	Yes
frontend-build	TypeScript typecheck and a production Vite build	Yes
docker-build	Builds both images so a broken Dockerfile fails CI	Yes
report	Publishes results back onto the pull request	No

Pull-request reporting

Two free actions, both authenticated with the token GitHub injects automatically — no third-party bot to install and no account to create.

- **publish-unit-test-result-action** — a check run with pass, fail and skip counts, and every failing test annotated inline on the diff.
- **sticky-pull-request-comment** — one comment carrying the overall status and coverage percentage, edited in place on each push rather than piling up.

> **The report job only runs on pull requests**
> It is gated on the event type, because a PR comment has nowhere to go on a plain push. A push shows the same information on the workflow summary page instead.

Running the workflows locally

The workflows are written to run under act. Steps that need the GitHub API — artifact upload and the PR comments — are skipped automatically when act is detected, so a local run exercises the real build and test steps without failing on calls it cannot make.

```
cp .github/act.secrets.example .secrets # required: .actrc points at it

act pull_request # the whole CI workflow
act -j backend-tests # one job
act -l # list what would run
```

Continuous deployment

Pushing to the main branch, or tagging a release, builds both images and publishes them to the GitHub Container Registry, then optionally calls a deploy hook and smoke-tests the health endpoint until the new revision responds. The deploy step skips cleanly when no hook is configured, so the workflow is not a permanent failure on a repository that is not yet deployed.

12 Deployment

How the server is set up

The backend image is built on python:3.13-slim. It installs the pinned requirements, copies the project, creates a non-root user and exposes port 8000. The entrypoint runs on every start and:

- waits for PostgreSQL, retrying for up to a minute, so a cold start has no ordering race;
- applies migrations;
- collects static files into a named volume;
- seeds demo data when enabled — the command is idempotent, so restarts are safe;
- hands over to gunicorn with three workers and two threads each.

The health endpoint backs the container healthcheck, and the frontend waits for the backend to report healthy, so nginx never starts in front of a dead upstream.

Local

```
docker compose up --build

# http://localhost:8080 the React application
# http://localhost:8080/accounts/ Django auth pages
# http://localhost:8080/admin/ Django admin
# http://localhost:8080/api/docs/ Swagger UI
# http://localhost:8025 Mailpit inbox
```

Hosting

Component	Options	Notes
Database	Neon, Supabase, Render PostgreSQL	Set DATABASE_URL
Backend	Render, Railway, Fly.io	Deploy the backend Dockerfile
Frontend	Vercel, Netlify, Render Static	Build command npm run build, output dist
Media	Cloudinary	Already configured
E-mail	Brevo, SendGrid, SES	Free tiers available

> **Prefer deploying the stack as one unit**
> On split origins the JWT login works fine, but the session-to-JWT handoff needs a cookie that survives cross-site, which means SameSite=None and Secure over HTTPS. Deploying the Compose stack as a single unit keeps the single origin and everything behaves exactly as it does locally.

Production checklist

- Generate a fresh SECRET_KEY and a strong database password.
- Set DEBUG=False and SECURE_SSL_REDIRECT=True.
- Set ALLOWED_HOSTS, CSRF_TRUSTED_ORIGINS and FRONTEND_URL to the real domain — FRONTEND_URL in particular, because emailed links are built from it.
- Point EMAIL_BACKEND at a real SMTP provider.
- Set SEED_DEMO_DATA=False once real data exists.

- Run `manage.py check --deploy`.

13 Configuration reference

Nothing environment-specific is hard-coded. Every value is read through typed environment variables with safe defaults, so the same image runs unchanged in development, Docker and production.

Variable	Default	Purpose
DEBUG	False	Never enable in production
SECRET_KEY	placeholder	Must be replaced when deploying
ALLOWED_HOSTS	localhost, backend, frontend	Comma separated
CSRF_TRUSTED_ORIGINS	localhost:8080	Required for the Django forms
DATABASE_URL	empty → SQLite	Compose builds it from the POSTGRES_* values
FRONTEND_URL	localhost:8080	Origin used to build emailed links
ACCESS_TOKEN_LIFETIME_MINUTES	60	JWT access lifetime
REFRESH_TOKEN_LIFETIME_DAYS	7	Rotated and blacklisted on use
SIGNUP_OPEN	True	False shows the sign-up-closed page
REQUIRE_EMAIL_VERIFICATION	False	True gates login behind confirmation
EMAIL_BACKEND / EMAIL_HOST	SMTP → Mailpit	Console backend outside Docker
MEDIA_STORAGE	local	local, cloudinary or s3
CLOUDINARY_*	empty	Cloud name, API key and secret
COMPANY_*	sample values	Printed on the challan PDF
SEED_DEMO_DATA	True	Entrypoint runs the demo seed
SECURE_SSL_REDIRECT	False	Enable behind HTTPS

Four example files ship with the repository and are safe to commit: the Compose environment, the backend environment, the frontend build variables and the act secrets. The real files they are copied to are all git-ignored. A dedicated environment guide documents where to obtain each credential.

14 Assumptions

Decisions taken where the brief was silent. Each is reversible, and the reasoning matters more than the choice.

1. Django instead of Node Requested explicitly by the reviewer.
2. One role per user A company of this size has staff sitting in one team. Multiple roles would need a many-to-many and a permission union.
3. Mobile number is unique per customer It is the field staff actually search by, and duplicate CRM records for one number are the classic data-quality problem. Relax the constraint if a shared office line must be allowed.
4. GST is mandatory for wholesale and distributor customers Those two invoice against a GSTIN; retail customers frequently do not have one.
5. New leads require a follow-up date A lead with no next action gets forgotten, which is the failure mode a CRM exists to prevent.
6. Self-service signup creates an active Sales user An administrator changes the role afterwards. Set SIGNUP_OPEN=False for an invite-only deployment.
7. Nothing is hard-deleted Customers become Inactive, products Archived, users Deactivated — challans and the stock ledger reference them for audit. Only draft challans can truly be deleted.
8. Cancelling a confirmed challan returns the stock in full Partial returns would need a separate sales-return document.
9. Prices exclude GST, computed per line from the product's rate No CGST/SGST/IGST split, which needs place-of-supply logic.
10. Line prices may be overridden on a challan Negotiated rates are normal in wholesale; the snapshot then keeps the agreed price.
11. Currency is INR and the timezone Asia/Kolkata Both configurable.
12. E-mail verification is off by default So the demo has no friction. Enable it and point at a real SMTP provider for production.

15 Known limitations

Stated plainly, because a case study that claims no weaknesses is not credible.

Limitation	Impact and what would fix it
No live deployment URL	The submission is the Docker stack plus documentation. Section 12 documents exactly how to deploy it.
No frontend unit tests	TypeScript in strict mode and 168 backend tests carry the correctness load. Component tests with Vitest and Testing Library would be the next addition.
Dashboard aggregates computed per request	Fine at this data volume; a large dataset would want caching or materialised counters.
Search is ILIKE-based	Good to a few hundred thousand rows. Beyond that, PostgreSQL full-text search or a trigram index.
No CSV import or export	A natural next step for both customers and products, and commonly expected in this domain.
No field-level audit history	Rows carry created_by, updated_by and timestamps, but not a full change log. django-simple-history would provide it.
Single-warehouse semantics	Location is a label on the product, not a separate balance per warehouse. Real multi-warehouse stock needs a per-location quantity table.
Session-to-JWT handoff assumes one origin	Cross-site cookies would need SameSite=None and Secure. See section 12.
Rate limiting only on login and password reset	Broader throttling would be sensible before exposing the API publicly.
Purchase orders and invoices not built	Deliberate — outside the four required modules. Stock IN movements already model goods receipt, so purchase orders would layer on top cleanly.

A Command reference

Running the stack

```
docker compose up --build # build and start everything
docker compose down # stop, keeping the database
docker compose down -v # stop and delete the database volume
docker compose logs -f backend # tail the backend logs

./run.sh # wrapper: WSL / Linux / macOS / Git Bash
.\run.ps1 # wrapper: Windows PowerShell
make up # wrapper: make
```

Database and demo data

```
docker compose exec backend python manage.py migrate
docker compose exec backend python manage.py makemigrations
docker compose exec backend python manage.py seed_demo
docker compose exec backend python manage.py seed_demo --reset
docker compose exec backend python manage.py createsuperuser
docker compose exec db psql -U opsportal -d opsportal
```

Tests and quality

```
docker compose exec backend python manage.py test
cd backend && coverage run manage.py test && coverage report
cd backend && ruff check . && ruff format --check .
cd frontend && npm run typecheck && npm run build
docker compose exec backend python manage.py check --deploy
```

Local development without Docker

```
conda create -n opsportal python=3.12 -y && conda activate opsportal
cd backend && pip install -r requirements-dev.txt
python manage.py migrate && python manage.py seed_demo
python manage.py runserver 0.0.0.0:8000

cd frontend && npm install && npm run dev # http://localhost:5173
```

Test logins

Role	Username	Password
Admin	admin	Admin@12345
Sales	sales	Sales@12345
Warehouse	warehouse	Warehouse@12345
Accounts	accounts	Accounts@12345

Repository documentation
README.md — setup and overview · docs/ARCHITECTURE.md — Mermaid diagrams · docs/ENVIRONMENT.md — every environment variable and where to obtain it · docs/OpsPortal.postman_collection.json — 67 API requests with assertions.